

MINI PROJECT REPORT

JAVA TO-DO LIST APPLICATION

Developed using Java with Methods and Data Structures

Project Type	Java Console-Based Mini Project
Primary Language	Java
Main Data Structure	ArrayList
Core Concepts	Methods, Class, Object, Loop, Switch, Conditional Statements
Input Handling	Scanner
Prepared For	Internship Project Submission
Submitted By	_____

A simple, beginner-friendly Java application that allows users to add, view, delete, and complete tasks through a menu-driven console interface.

1. Project Overview

The Java To-Do List Application is a simple console-based task management program. Its purpose is to demonstrate how basic Java programming concepts can be combined to build a useful mini application. The user can create tasks, see all saved tasks, delete a task, and mark a task as completed.

The project follows the internship instruction to develop a basic To-Do List using functions (methods) and a suitable data structure. In this project, an ArrayList is used to store task objects because the number of tasks can change while the program is running.

2. Objective

- Develop a simple To-Do List application using Java.
- Understand and use methods to divide the program into smaller tasks.
- Use ArrayList to store and manage multiple task objects.
- Use a class and objects to represent individual tasks.
- Accept user input using the Scanner class.
- Apply loops, switch statements, if-else conditions, and boolean values.
- Provide a clear menu-driven interface that is easy for a beginner to understand.

3. Main Features

Feature	What it does
Add Task	Creates a new task and stores it in the ArrayList.
Delete Task	Removes a selected task using its task number.
Display Tasks	Shows all stored tasks with their current status.
Mark Complete	Changes a task's status from Pending to Completed.
Exit	Ends the program safely and closes the Scanner.

4. Technologies and Concepts Used

Technology / Concept	Easy Explanation
Java	The programming language used to build the application.
ArrayList	A resizable list used to store all Task objects.
Class	A blueprint used to define what a Task contains.
Object	An individual Task created from the Task class.
Methods	Separate blocks of code for adding, deleting, displaying, and completing tasks.
Scanner	Reads the user's keyboard input.
boolean	Stores whether a task is completed or still pending.
if-else	Checks conditions such as whether a task number is valid.
for loop	Visits each task in the ArrayList when displaying the list.
switch	Chooses an action based on the user's menu choice.
do-while	Keeps the menu running until the user selects Exit.

5. Data Structure: ArrayList

ArrayList is the main data structure used in this project. It is similar to a list that can grow or shrink during program execution. When the user adds a task, a new Task object is added to the ArrayList. When the user deletes a task, that object is removed.

Easy way to remember: Think of ArrayList as a flexible notebook. You can keep adding new pages (tasks), remove a page, and read all pages one by one.

6. Task Class and Object

The program contains an inner class named Task. Each Task object stores two pieces of information: the task name and whether the task is completed.

name stores the text of the task, such as "Study Java". completed is a boolean value. It is initially false, which means Pending. When the user completes the task, it becomes true and the program displays Completed.

7. Methods Used in the Project

Method	Purpose
addTask()	Creates a Task object and adds it to the ArrayList.
deleteTask()	Checks the task number and removes the selected task.
displayTasks()	Loops through the ArrayList and prints every task and its status.
markTaskComplete()	Checks the task number and changes completed from false to true.
main()	Starts the program, displays the menu, accepts input, and calls the required methods.

8. Program Working

Step 1: The program starts from the main() method.

Step 2: A Scanner object reads the user's input.

Step 3: A menu is displayed with five options.

Step 4: The switch statement identifies the selected option.

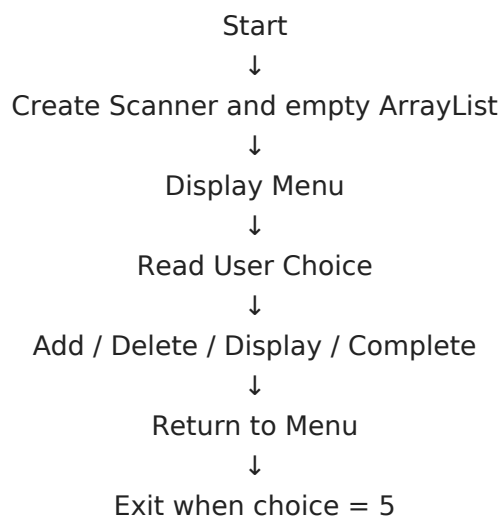
Step 5: The corresponding method is called.

Step 6: The selected operation is performed on the ArrayList.

Step 7: The menu appears again because of the do-while loop.

Step 8: The program ends when the user selects option 5.

9. Program Flow



10. Complete Source Code

The following is the complete Java source code used to implement the project. It can be copied directly into a file named `ToDoList.java`.

Source Code — Lines 1-38

```
import java.util.ArrayList;
import java.util.Scanner;

public class ToDoList {

    // ArrayList to store all tasks
    static ArrayList<Task> tasks = new ArrayList<>();

    // Class representing a task
    static class Task {
        String name;
        boolean completed;

        Task(String name) {
            this.name = name;
            this.completed = false;
        }
    }

    // Method to add a new task
    public static void addTask(String taskName) {
        tasks.add(new Task(taskName));
        System.out.println("\nTask added successfully!");
    }

    // Method to delete a task
    public static void deleteTask(int taskNumber) {
        if (taskNumber >= 1 && taskNumber <= tasks.size()) {
            tasks.remove(taskNumber - 1);
            System.out.println("\nTask deleted successfully!");
        } else {
            System.out.println("\nInvalid task number!");
        }
    }

    // Method to display all tasks
    public static void displayTasks() {
        if (tasks.isEmpty()) {
```

Source Code — Lines 39-76

```
        System.out.println("\nNo tasks available.");
        return;
    }

    System.out.println("\n===== TO-DO LIST =====");

    for (int i = 0; i < tasks.size(); i++) {
        Task task = tasks.get(i);

        String status = task.completed ? "[Completed]" : "[Pending]";

        System.out.println((i + 1) + ". " + task.name + " " + status);
    }

    System.out.println("=====");
}

// Method to mark a task as complete
public static void markTaskComplete(int taskNumber) {
    if (taskNumber >= 1 && taskNumber <= tasks.size()) {
        tasks.get(taskNumber - 1).completed = true;
        System.out.println("\nTask marked as completed!");
    } else {
        System.out.println("\nInvalid task number!");
    }
}

// Main method
public static void main(String[] args) {

    Scanner scanner = new Scanner(System.in);

    int choice;

    System.out.println("=====");
    System.out.println("        JAVA TO-DO LIST");
    System.out.println("=====");
```

Source Code — Lines 77-114

```
do {
    System.out.println("\n----- MENU -----");
    System.out.println("1. Add Task");
    System.out.println("2. Delete Task");
    System.out.println("3. Display Tasks");
    System.out.println("4. Mark Task as Complete");
    System.out.println("5. Exit");
    System.out.println("-----");

    System.out.print("Enter your choice: ");
    choice = scanner.nextInt();
    scanner.nextLine();

    switch (choice) {

        case 1:
            System.out.print("Enter task name: ");
            String taskName = scanner.nextLine();

            addTask(taskName);
            break;

        case 2:
            displayTasks();

            System.out.print("Enter task number to delete: ");
            int deleteNumber = scanner.nextInt();

            deleteTask(deleteNumber);
            break;

        case 3:
            displayTasks();
            break;

        case 4:
            displayTasks();
```

Source Code — Lines 115-133

```
        System.out.print("Enter task number to mark as complete: ");
        int completeNumber = scanner.nextInt();

        markTaskComplete(completeNumber);
        break;

    case 5:
        System.out.println("\nThank you for using the To-Do List!");
        break;

    default:
        System.out.println("\nInvalid choice! Please try again.");
    }

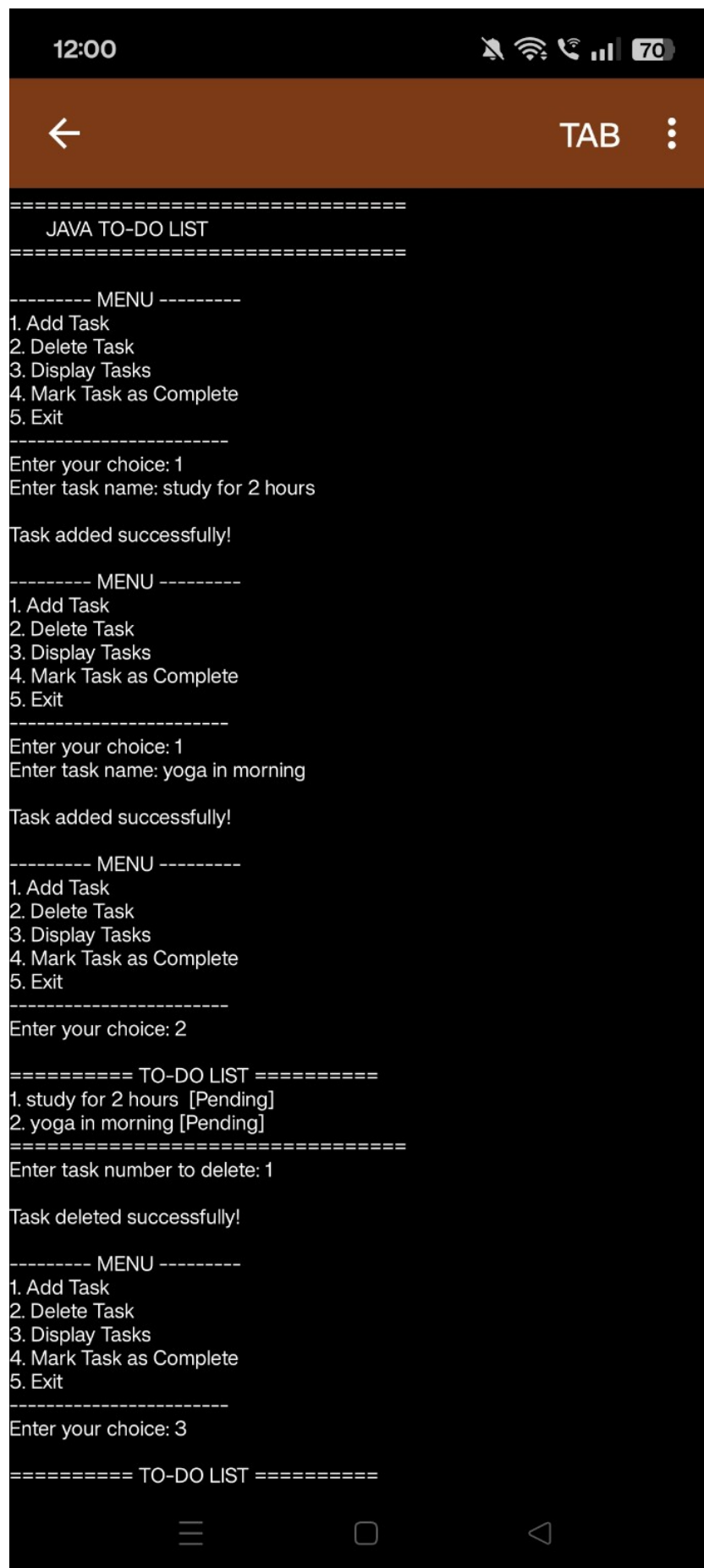
    } while (choice != 5);

    scanner.close();
}
```


11. Output and Testing

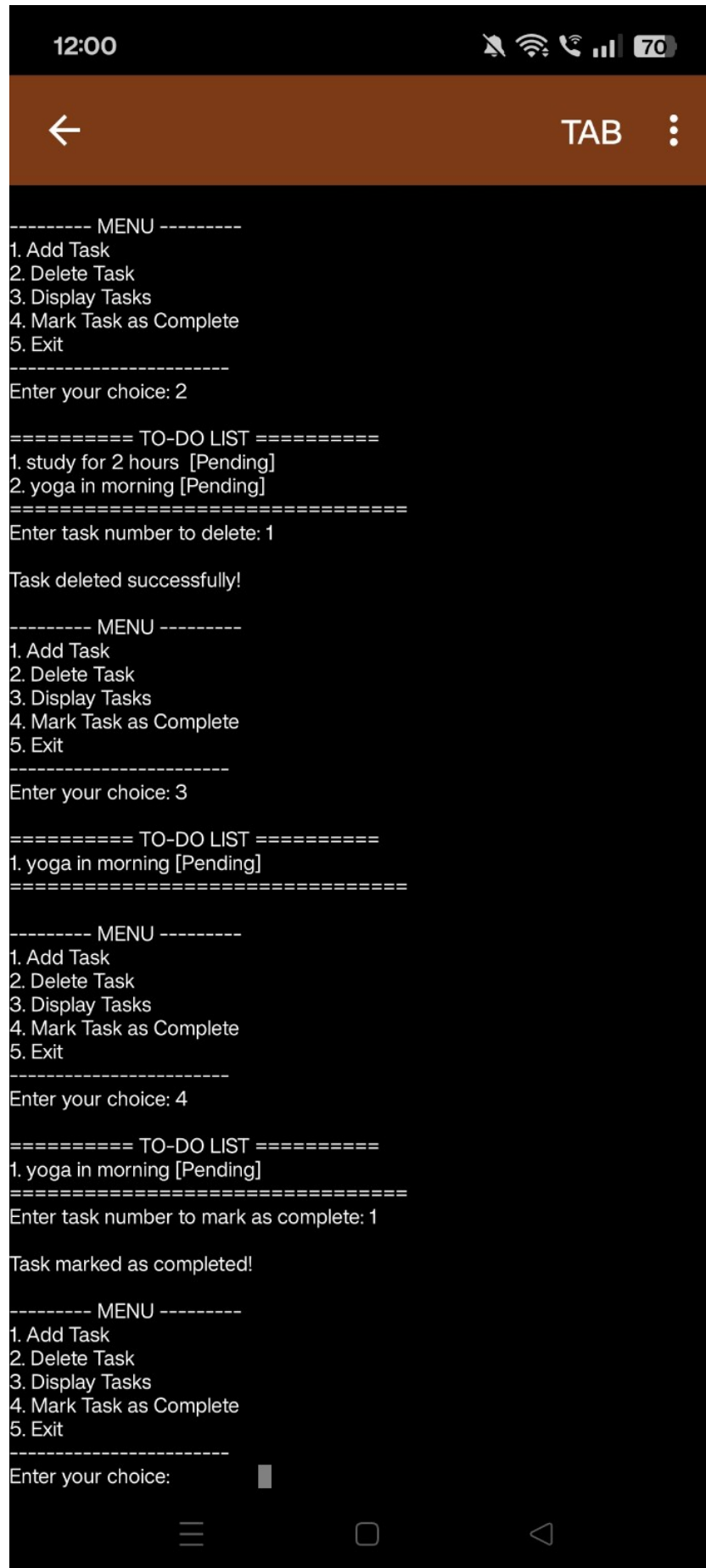
The application was executed successfully and tested using the required operations. The screenshots below show actual console output from the program.

Output Screenshot 1 — Adding and Deleting Tasks



This screenshot demonstrates the Add Task operation, followed by displaying the task list and deleting a selected task successfully.

Output Screenshot 2 — Displaying and Completing a Task



This screenshot demonstrates the Display Tasks operation and the Mark Task as Complete operation. The selected task changes from [Pending] to [Completed].

12. Test Cases

Test	Input	Expected Result	Observed Result
Add task	1 + task name	Task is added	Passed
Display tasks	3	All tasks are shown	Passed
Delete task	2 + task number	Selected task is removed	Passed
Complete task	4 + task number	Status becomes Completed	Passed
Exit	5	Program ends	Passed

13. Advantages of the Project

- Simple and easy-to-use menu-driven interface.
- Uses an appropriate data structure for a dynamic list of tasks.
- Uses separate methods, making the code organized and easier to maintain.
- Demonstrates important beginner-level Java concepts in one project.
- The project can be expanded later into a graphical or web-based application.

14. Limitations

- Tasks are stored only while the program is running; there is no permanent database or file storage.
- The current interface is console-based rather than graphical.
- The program does not include user accounts or multiple users.
- Task information is limited to a name and completion status.

15. Future Enhancements

- Save tasks permanently using a text file, JSON file, or database.
- Add task priorities such as High, Medium, and Low.
- Add due dates and reminders.
- Add an option to edit an existing task.
- Create a graphical user interface using Java Swing or JavaFX.
- Develop a web version with a Java backend such as Spring Boot.
- Add search and filtering features.

16. Conclusion

The Java To-Do List Application successfully demonstrates the use of Java methods and data structures to solve a simple real-world problem. The project uses an ArrayList to manage Task objects and provides separate methods for adding, deleting, displaying, and completing tasks. It also demonstrates Scanner input, classes and objects, boolean values, loops, switch statements, and conditional checks.

Overall, this project provides a strong beginner-level example of how basic Java concepts work together to create a functional application. It also provides a good foundation for future improvements such as file storage, databases, graphical interfaces, and web applications.

Project Status: Successfully Implemented and Tested

17. Viva / Interview Quick Explanation

What is this project? A Java-based console To-Do List used to add, display, delete, and complete tasks.

Why ArrayList? Because the number of tasks can change dynamically. ArrayList makes adding and removing elements convenient.

What is a Task class? It is a blueprint for a task. Each object contains a name and a completed status.

Why boolean? It stores two states: false for Pending and true for Completed.

Why methods? Methods divide the program into small reusable parts and make the code easier to understand.

What does Scanner do? It takes input from the user through the keyboard.

Why do-while? The menu should appear at least once and continue until the user selects Exit.

What does `tasks.remove(taskNumber - 1)` mean? Users see task numbers starting from 1, while ArrayList indexes start from 0, so 1 is subtracted.